Pathways to Building an Open-Source Aerospace Technology Sharing Ecosystem and Deep-Space Simulation Platform Informed by the History of Computer Development

Y. X. Tan*

August 2026

Abstract

Space engineering is characterized by high technical barriers, heavy investment, long development cycles and lengthy collaboration chains. It has long been dominated by state-led, institutionally closed development, which leads to duplicated R&D, solidified knowledge barriers and limited participation of small innovators. By contrast, over the past decades the computer industry has pushed the question of "how to make extremely complex systems efficient, reliable and usable by everyone" to its limits, developing a series of deep architectural ideas including open-source collaboration, hardware–software decoupling, simulation and virtualization, microservice architecture, automated test-and-delivery pipelines and open commercial ecosystems. Taking the history of computer development as a reference frame and using historical comparison, case analysis and publicly available statistics (GitHub, npm/PyPI, the Linux kernel, CubeSat launches and the global satellite industry), this paper distills the evolution and common laws of these ideas and proposes an overall scheme for building an open-source aerospace technology sharing ecosystem and a deep-space simulation platform. At the architecture level, it proposes a microservice-based spacecraft architecture and a universal spacecraft operating system (Space OS) that decouple attitude and orbit control, thermal control, communication and payload modules into independent services for cross-hardware reuse and single-point-fault resistance. At the development-process level, it proposes a "DevSpaceOps" model in which attitude and orbital algorithms contributed by open-source developers are automatically validated through thousands of extreme-condition simulations on a cloud-based digital twin before being merged into the mainline. At the ecosystem-reuse level, it envisions a Space Package Registry (analogous to npm/PyPI) for the standardized reuse of Kalman-filter orbital algorithms, MPPT algorithms and even CubeSat structural CAD models. At the security level, it introduces zero-trust architecture and Byzantine fault tolerance to safeguard the core aerospace safety baseline against unverified code under globally distributed contributions. At the sustainability level, it demonstrates a dual-licensing and commercial open-source synergy to solve the funding

*Corresponding author, E-mail: yxtan5022@gmail.com. Affiliation: TBD.

problem. The study concludes that drawing on the computer industry's open-source pathway can effectively lower the barrier to aerospace technology innovation, accelerate deep-space exploration iteration and talent cultivation, and provide ecosystem-level support for the high-quality development of China's aerospace sector.

Keywords: history of computer development; open-source ecosystem; aerospace technology sharing; deep-space simulation platform; microservice architecture; DevSpaceOps; technology standardization; digital twin

1 Introduction

1.1 Background and Research Question

Space engineering is a typical strategic high-technology field, characterized by high technical barriers, heavy investment, long development cycles and lengthy collaboration chains. Historically, spacecraft are developed in a state-led, vertically integrated and relatively closed manner: from requirement demonstration and overall design, through subsystem development and satellite-level integration, to on-orbit operation, every stage is highly coupled, and technology, data and infrastructure are concentrated in a small number of large institutions. This model has played an irreplaceable role in accomplishing national missions, but it also gives rise to a series of challenges. First, duplicated R&D is prominent: homogeneous subsystems are repeatedly developed across many spacecraft models, and knowledge is hard to accumulate and reuse. Second, the "one satellite, one customized system" phenomenon is widespread: aerospace software is deeply bound to hardware, and almost every satellite is built from scratch, keeping cost and development cycles high. Third, the barrier to participation is too high for universities, small and medium-sized enterprises and individual developers, limiting innovation vitality. Fourth, emerging fields such as deep-space exploration require fast technological iteration, which the traditional closed model cannot sustain in terms of frequent verification and talent cultivation.

The closed nature of aerospace contrasts sharply with the computer industry. Since the middle of the twentieth century, computing has undergone a historic transition from the vertically integrated, closed system of the mainframe era to an open-source and open ecosystem. Open-source projects such as Linux, GNU and BSD established a globally collaborative software development model; open standards such as TCP/IP and POSIX enabled interoperability and reuse across devices and software; from monolithic architectures to microservices and from bare metal to virtualization and containerization, complex systems have been continuously decomposed into standard modules that evolve independently; and time-sharing systems, emulators and digital twins have dramatically reduced the cost of physical verification. In a few decades, the computer industry has almost fully answered the question of how to make extremely complex systems efficient, reliable and usable by everyone.

This raises the research question of this paper: can the deep architectural ideas of computer history be systematically transferred to the aerospace domain, so as to build an open-source aerospace technology sharing ecosystem and a deep-space simulation platform, thereby solving the problems of high entry barriers and closed development in aerospace innovation? This paper attempts to answer this question.

1.2 Significance

At the theoretical level, this paper introduces software engineering, open-source governance and open innovation theory into aerospace engineering research, constructs a concept-mapping framework from "computer prototypes" to "aerospace forms", provides a new analytical perspective for knowledge sharing and ecosystem building in aerospace, and enriches the application of open-source theory to hard technology and complex systems engineering.

At the practical level, the microservice-based spacecraft architecture, Space OS, DevSpaceOps and Space Package Registry proposed in this paper offer operational path references for lowering the barrier to aerospace R&D, accelerating technological iteration in deep-space exploration, alleviating talent shortages and promoting industry–academia–research collaboration, and have practical guiding value for building the open-source aerospace ecosystem.

1.3 Research Approach and Methods

This paper mainly adopts the following methods. (1) *Historical comparison.* Taking the evolution of the computer industry from closed to open as the reference frame, it compares the existing spacecraft development model and analyzes the similarities and differences between the two kinds of complex systems in organizational structure, technical architecture and innovation ecosystem. (2) *Case analysis.* It selects representative cases such as Linux/GNU, BSD, Docker, npm/PyPI, Red Hat and Android, examines the mechanisms behind their success, and compares them with existing aerospace practices such as the NASA open-source program, ESA open-source projects and open-source CubeSat networks. (3) *Concept mapping.* It establishes a one-to-one correspondence among "computer prototype, corresponding modern technology, proposed aerospace form, and core value", forming a concept-mapping table that serves as the pivot of the whole argument. (4) *Statistical data analysis.* It uses public statistics from GitHub, npm/PyPI, the Linux kernel, the Nanosats database and the SIA reports to provide quantitative evidence for the ecosystem vision.

1.4 Contributions

The contributions of this paper are twofold. First, it is the first attempt to systematically transfer to the aerospace domain, as an integrated whole, the deep ideas of computer history including microservice architecture, CI/CD pipelines, package-manager ecosystems, dual-licensing business models, zero trust and Byzantine fault tolerance. Second, it proposes and defines original concepts including the microservice-based spacecraft architecture, Space OS,

DevSpaceOps and Space Package Registry, and constructs a relatively complete blueprint of the open-source aerospace ecosystem.

2 Literature Review and Theoretical Foundations

2.1 The History of the Computer Industry

Historical studies of the computer industry reveal the path of information-technology systems from closed to open. The early IBM System/360 established a business model in which hardware and operating systems were vertically bound, with software as an appendix to hardware; meanwhile, the counterculture and hacker ethics that arose in the 1960s–70s promoted the free dissemination of software knowledge [6, 1]. As UNIX was born at Bell Laboratories and spread widely in universities and research institutions, its permissive licensing and portable design nurtured the later free-software and open-source movements. In the 1980s, the TCP/IP protocol stack was standardized through DARPA funding and the public RFC document process, breaking the segmentation of proprietary vendor network protocols and laying the foundation for internet interoperability; the POSIX standard unified operating-system interfaces [7, 8], enabling application portability across vendor hardware and driving hardware–software decoupling. Scholars generally agree that standardization and openness were the key reasons the computer industry moved from vertical integration to horizontal division of labor and achieved exponential growth.

2.2 Open-Source Communities and Open Innovation Theory

Research on open-source communities provides a mature framework for understanding distributed collaboration. Raymond summarized the success of Linux as a contrast between the "cathedral" and the "bazaar" development models, noted that "given enough eyeballs, all bugs are shallow", and distilled governance principles such as "release early, release often" [1]. Stallman articulated the philosophy of free software on moral and political grounds, arguing that source code, documentation and ideas should be freely shared [2]. Fogel systematically discussed the governance mechanisms of sustainable open-source projects, including version control, contribution guidelines, maintainer tiers and conflict resolution [3]. In the economics of innovation, von Hippel proposed the theory of "user innovation", holding that lead users often identify needs and build prototypes before manufacturers [4]; Chesbrough proposed the "open innovation" paradigm, arguing that firms should use both internal and external knowledge resources [5]. These theories provide the scholarly foundation for our argument on whether open source can support a safety-critical, high-reliability domain such as aerospace.

2.3 Open-Source Aerospace and Digital Twin Research

Openness in the aerospace domain started later but has accumulated some foundations. NASA has long published a large body of software and data assets; its open-source software catalog covers mission planning, orbital mechanics, image processing and other categories, and multiple projects are hosted on GitHub [15]. Research by NASA and the U.S. Air Force proposed the "digital twin" paradigm for the whole lifecycle of vehicles, advocating real-time mirrored virtual models spanning design, manufacturing and operations [12, 13]. ESA has also established open-source policies and open toolchains, promoting community maintenance of satellite control and mission analysis software [16]. At the micro-nano-satellite level, the CubeSat standard was proposed by U.S. universities and rapidly internationalized [14]; large open ground-station networks (e.g., TinyGS) and open-source flight-control firmware (e.g., ArduPilot, MAVLink) have enabled low-cost participation in low-Earth-orbit activities [18, 19, 20]. Domestically, concepts such as software-defined satellites and on-orbit reconfiguration [23] reflect the exploration of "software-defined, platform-based, ecosystem-oriented" development in the aerospace community. Overall, existing research focuses on specific open-source projects or individual technologies, and still lacks a theoretical framework that systematically builds an open-source aerospace ecosystem from the overall laws of computer history.

2.4 Gap Statement

In summary, the existing literature supports this study in three ways: first, mature theories of open-source governance and open innovation can be directly transferred; second, computer history provides sufficient empirical evidence that openness promotes innovation; and third, aerospace open-source practice and digital-twin technology demonstrate the feasibility of such a transfer. However, there are obvious gaps: first, no attempt has systematically introduced the deep architectural ideas of computer history—microservices, CI/CD, package managers, dual licensing, zero trust and fault tolerance—into the aerospace domain as a whole; second, aerospace open-source research is mostly project-based and lacks an overall architecture design at the "ecosystem" level; and third, there is no dedicated treatment of funding sources and commercial sustainability for open-source aerospace, nor support from public statistical data. This paper fills these gaps by constructing a complete scheme for an open-source aerospace technology sharing ecosystem and a deep-space simulation platform.

3 Lessons from the History of Computer Development

This chapter systematically reviews the deep architectural ideas of computer history along four main lines and distills common laws and a concept-mapping table at the end.

3.1 Open-Source Movement and Free Software: Reconstructing Collaboration

The open-source movement changed how complex software is produced and organized. In 1983 Stallman launched the GNU project, advocating free sharing of software [2]; in 1991 the Linux kernel released by Linus Torvalds, combined with the GNU toolchain, formed the first fully open modern operating system capable of free evolution [1]. The essence of open-source collaboration is not "free of charge" but the delegation of the right to know, modify and redistribute development to every participant, thereby aggregating global intelligence in a distributed manner. Git and GitHub further lowered the participation barrier: anyone can fork, commit and open pull requests, and maintainers review and merge them into the mainline [21]. This "open submission, centralized adjudication" model guarantees code quality while recording and rewarding contributions, forming a virtuous cycle of large-scale collaboration. Its lesson is that the evolution of complex systems need not depend on total control by a single authoritative organization, but rather on public rules, transparent processes and accumulated trust.

3.2 Standardization: From Proprietary to Open Standards

Standardization is the key lever through which the computer industry moved from fragmentation to interconnection. TCP/IP replaced vendor-proprietary protocols through the public RFC document system and the open IETF process [7], achieving the global unification of the internet; the POSIX interface standard freed application developers from underlying hardware differences, making "write once, run anywhere" possible [8]. The openness of a standard determines the boundary of its ecosystem: the more open the interface, the more third parties it attracts, forming positive feedback. For aerospace, the lesson is that openness at the interface and protocol level often has more leverage than openness of implementation—once a standard is established, participants can freely compete and combine on a common baseline.

3.3 Modularity and Middleware: Decomposition of Complex Systems

Computer systems evolved from early monolithic programs to layered, modular, service-oriented structures. The hardware abstraction layer (HAL) and device-driver mechanisms hide underlying differences, and the operating system acts as "middleware" bridging layers. Since the 1990s, enterprise software has shifted from heavyweight monoliths to service-oriented architectures and then to microservices [9]: complex systems are decomposed into many small services that communicate through standard APIs and can be independently deployed and evolved. Container technology (e.g., Docker) standardized the packaging, distribution and isolation of services [11]; the failure of one module no longer takes down the whole system, and versions can iterate independently at the module granularity. The core idea of this evolution is "decoupling": through clear boundary definitions, system complexity

is decomposed into structures that are locally autonomous and globally controlled.

3.4 Simulation and Virtualization: Low-Cost Iterative Validation

From the time-sharing systems of the mainframe era, through instruction-level emulators such as QEMU and software-in-the-loop (SITL) testing, to cloud-native computing and digital twins [12, 13], virtualization has always answered one question: how to obtain verification capability close to that of the physical world at a cost far below physical experimentation. Digital twins go further by binding virtual models to the real-time state of physical entities, supporting a closed loop across design, verification and operations. The lesson is that high reliability need not be achieved only by "building more prototypes and running more tests"; through high-fidelity simulation, automated testing and fault injection, verification cost can be reduced by orders of magnitude while test coverage far exceeds what manual effort can reach.

3.5 The Symbiosis of Open Source and Commerce

Open source and commerce are not opposed. Red Hat built enterprise-grade support and services on open-source Linux and was acquired by IBM for billions of dollars; MySQL built a vast ecosystem through a dual-licensing model combining an open-source community edition and a commercial value-added edition; Android aggregated hardware vendors on an open-source base while monetizing through ecosystem services. The common pattern is that open source provides widely trusted infrastructure, while commerce provides certainty, services and premium capabilities on top of it, forming a closed loop of "open source attracts, commerce feeds". This model answers the sharpest question of all: where does the money for open-source aerospace come from?

3.6 Quantitative Evidence: The Scale Effect of Open Ecosystems

The laws above are not only logically self-consistent but also supported by macro-level data. Regarding collaborative platforms, GitHub surpassed 100 million registered developers in early 2023; in 2024 developers worldwide made 5.2 billion contributions across 518 million open-source, public and private projects, of which roughly 1 billion contributions went to open-source and public repositories [24]. In package ecosystems, the number of npm packages grew from about 462,000 in April 2017 to about 4.875 million in October 2024, PyPI hosts about 635,000 packages, and the total annual request volume of major package registries reached about 6.689 trillion in 2023 [25]. The Linux kernel releases a major version roughly every ten weeks, with about 15,000 commits and about 2,000 developers per release, of which about 250 are newcomers; in 2023 more than 5,000 contributors participated in kernel development over the whole year [26].

By comparison, openness in the aerospace domain has also shown a scale effect: according to the Nanosats database, by September 2024, 2,714 nanosatellites had been launched

worldwide (including 2,505 CubeSats), and a record 390 nanosatellites were launched in 2023 alone; the cumulative number of CubeSats passed the second thousand in only about four years, whereas the first thousand took about sixteen years (2003–2018), with 88 countries and more than 600 organizations involved [27]. In the commercial satellite industry, the SIA reports that 2023 global satellite industry revenue reached about USD 285 billion, accounting for 71% of the global space economy (about USD 400 billion); active satellites in orbit totaled 9,691, up 361% from 2019; and 2,781 commercial satellites were deployed in the year, up 20% year on year [28]. NASA has also publicly released hundreds of software programs and code repositories through its Software Release Authority (SRA) [15]. These figures show that openness and standardization have brought order-of-magnitude increases in the number of participants, the scale of knowledge assets and the speed of iteration in both the computer and aerospace industries, providing empirical support for the ecosystem vision of this paper. Key indicators are summarized in Table 1.

Table 1: Quantitative evidence of the scale effect of open ecosystems

Dimension	Computer open ecosystem	Open-source aerospace practice
Collaboration scale	GitHub: 100M+ developers, 518M+ projects; 5.2B contributions in 2024 [24]	2,505 CubeSats launched, 88 countries, 600+ organizations [27]
Components & assets	npm ∼4.875M packages, PyPI ∼635K packages; ∼6.689T requests in 2023 [25]	NASA released hundreds of open-source software programs [15]
Iteration speed	Linux: major release every ∼10 weeks, ∼15K commits, ∼2,000 developers each [26]	Second thousand CubeSats in ∼4 years (first thousand took ∼16 years) [27]
Economic scale	—	2023 satellite industry revenue ∼USD 285B (71% of space economy); 9,691 active satellites (+361% in 5 years) [28]

3.7 Common Patterns

Synthesizing the six paths above, six common laws can be distilled: (1) *openness and sharing*—the free flow of knowledge is the precondition for ecosystem growth; (2) *standards first*—open interfaces have more leverage than open implementations; (3) *layered decoupling*—transform system complexity into independently evolvable module boundaries; (4) *simulation first*—low-cost verification supports high-reliability delivery; (5) *security as a safety net*—distributed collaboration must be bounded by trust mechanisms and fault-tolerant design; and (6) *commercial feedback*—a sustainable ecosystem requires a self-funded business loop.

3.8 Concept Mapping

Mapping these lessons to the aerospace domain yields the concept-mapping table shown in Table 2, which serves as the pivot of the subsequent chapters.

Table 2: Concept mapping from computer history to the open-source aerospace ecosystem

Prototype	Modern computing	Proposed aerospace form	Core value
Open-source collaboration	Linux/Git/GitHub	OpenSpace R&D ecosystem	Break monopolies, lower entry barriers
H/W–S/W decoupling	POSIX/HAL	Spacecraft OS (Space OS)	Avoid one-satellite custom builds; cross-hardware reuse
Virtualization	QEMU/SITL/game engines	Deep-space digital twin & simulation apps	Verify algorithms without physical hardware
Modular services	Docker/microservices	Microservice satellite/payload architecture	Fault isolation and module replacement
Delivery pipelines	CI/CD automation	DevSpaceOps cloud simulation verification	Automated reliability testing in extreme space
Ecosystem reuse	npm/PyPI package managers	Space Package Registry (Space PM)	Avoid re-inventing the wheel, accelerate iteration
Open business loop	Red Hat/MySQL/Android dual licensing	Open base + commercial services	Sustainable funding for open-source aerospace
Fault tolerance	BFT consensus/zero trust	Zero-trust deep-space computing network	Community code cannot break the core safety baseline

4 Architecture of the Open-Source Aerospace Ecosystem

4.1 Overall Architecture Blueprint

Following the layered structure of "community–standards–platform–applications" in the computer industry, this paper proposes an overall architecture for the open-source aerospace technology sharing ecosystem, from bottom to top consisting of four layers: (1) the *community layer*, comprising global developers, research institutions, commercial companies and enthusiasts, who produce code, data and knowledge; (2) the *standards layer*, comprising open interfaces, data-exchange protocols and quality specifications that form the interoperability baseline; (3) the *platform layer*, comprising public infrastructure such as the universal spacecraft operating system (Space OS) and the deep-space simulation platform, which hides hardware differences above and provides a unified runtime environment below; and (4) the *application layer*, comprising algorithms, payloads and services for specific missions (low-Earth-orbit constellations, deep-space exploration, on-orbit servicing, etc.). The four layers are loosely coupled through standard interfaces, and the evolution of any layer does not force breakage of the others—an embodiment of modular thinking.

4.2 Open Collaboration Layer: The OpenSpace R&D Ecosystem

Drawing on the governance experience of Linux and GitHub, the OpenSpace R&D ecosystem should establish open and transparent collaboration mechanisms: first, define contribution guidelines and coding standards, and clarify the fork–submit–review–merge workflow; second, implement tiered maintainer governance, with core maintainers, subsystem maintainers and general contributors forming a hierarchical trust structure; third, democratize technical decisions through public issue discussions, design documents and a technical committee; and fourth, build contribution records and an honor system so that individual contributions gain accumulative reputational returns. The core problem this layer solves is: how can developers distributed around the world trust one another and collaborate continuously around a common aerospace goal?

4.3 Standardization Layer: Open Interfaces and Protocols

Standardization is the foundation of ecosystem interconnection. Drawing on POSIX and TCP/IP, the aerospace domain should prioritize four categories of open specifications: (1) *hardware interface standards*, normalizing electrical and data interfaces for on-board computing, communication and power so that components from different vendors are plug-and-play; (2) *software interface standards*, unifying data models and invocation interfaces (similar to service APIs) among attitude-and-orbit-control, thermal-control, communication and payload services to achieve "cross-hardware reuse of applications"; (3) *data-exchange protocols*, normalizing the encoding and transmission formats of telemetry, telecommand, orbit prediction and scientific data, extending the CCSDS standards of ground control toward more

openness; and (4) *testing and quality specifications*, defining benchmark scenarios, fault-injection sets and acceptance criteria for simulation-based verification. Once standards are widely adopted, participants can freely compete and combine on a common baseline, fundamentally alleviating the "one satellite, one customized system" problem.

4.4 Platform Layer: The Universal Spacecraft Operating System (Space OS)

Space OS is the core platform proposed by analogy with operating systems and microservice thinking. Its basic idea is to decouple the traditionally highly coupled satellite into functional modules—attitude and orbit control, thermal control, communication, power and payload—each running logically as an independent service and communicating through a unified kernel and message bus. Space OS provides the following capabilities: (1) *hardware abstraction*: a driver layer hides hardware differences across platforms (CubeSats, small satellites, deep-space probes), so the same attitude-control or orbit algorithm can be ported to different satellites; (2) *service isolation*: functional services start, upgrade and recover independently, and the anomaly of one service does not affect other services or overall spacecraft safety; (3) *on-orbit reconfiguration*: following software-defined thinking, functional modules can be loaded, updated and replaced on orbit, extending mission lifetime and enhancing flexibility; and (4) *safety baseline*: a minimum safe mode (Safe Mode) is built in, and no matter how upper-layer applications evolve, the protected core safeguard logic always remains. The core problem this layer solves is transforming "integrated spacecraft" into "platform plus services", so that the community can contribute to and iterate on a single microservice module (e.g., a "radiation-tolerant on-orbit data compression algorithm") without understanding the whole satellite.

4.5 Reuse Layer: The Space Package Registry

By analogy with package managers such as npm/PyPI/cargo, this paper envisions a Space Package Registry that publishes aerospace software and hardware assets in a versioned, dependable and auditable manner, including but not limited to: orbital-dynamics and Kalman-filter orbit-prediction algorithms, maximum-power-point-tracking (MPPT) control code for solar panels, attitude determination and control algorithms, deep-space communication codecs, 3D-printed CAD models of satellite structures and CubeSat boards, and simulation environment models. The registry should provide standardized version management and dependency resolution (answering "which version to depend on and which platforms are compatible"), quality review and signature verification, and channels for security vulnerability disclosure. Through this registry, projects worldwide can reference mature components as easily as "pip install", converting the cost of repeatedly reinventing the wheel into a combined effort focused on a small number of high-quality components, thereby greatly accelerating technological iteration.

5 Building the Deep-Space Simulation Platform

5.1 Deep-Space Digital Twin Environment

Deep-space missions (beyond the Moon, asteroid exploration, interplanetary flight) feature extreme environments, long communication delays and the impossibility of full-scale ground reproduction. The cost of physical experimentation is prohibitive, so simulation-based verification is a necessity. This paper advocates building a high-fidelity deep-space digital twin environment drawing on the simulation ideas of QEMU, software-in-the-loop (SITL) and game engines, mainly including four kinds of models: (1) *orbital and gravitational models*, covering the two-body problem, multi-body gravitation, solar radiation pressure and higher-order gravitational perturbations, supporting orbital simulation in Earth–Moon and interplanetary scenarios; (2) *deep-space environment models*, modeling solar wind, galactic cosmic rays, radiation belts, micrometeoroids and space debris, and their effects on electronics and structures; (3) *unit and subsystem models*, building functional-level and fault-level models of on-board computers, attitude sensors, actuators and communication links; and (4) *fault-injection sets*, defining standardized injection scenarios of extreme conditions such as sensor drift, actuator jamming, single-event upsets, communication loss and sudden solar storms. The key to a digital twin is not the accuracy of any single model but the consistency, reproducibility and auditability of model composition, providing the community with a trusted "virtual test range".

5.2 DevSpaceOps: Cloud-Simulation-Driven High-Reliability R&D

Aerospace engineering demands extremely high reliability; traditional practice relies on documentation review, ground testing and repeated trials, which is slow and costly. Drawing on CI/CD and DevOps practice in the software industry [10], this paper proposes the concept of "DevSpaceOps": embedding cloud-based digital-twin simulation into the delivery pipeline of open-source collaboration so that every contribution is automatically verified with high coverage. The typical workflow is as follows: (1) a developer submits attitude-control, orbital or other algorithm code to the registry or code repository; (2) the platform automatically triggers compilation, static analysis and unit tests; (3) in the digital twin environment, a predefined number of extreme-condition simulations are run automatically, covering sudden solar storms, sensor faults, single-event upsets and communication loss—"thousands of extreme-state tests"; (4) test pass/fail is decided according to predefined acceptance criteria (convergence, stability and safety metrics); and (5) only after passing can the code be merged into the mainline, with a traceable simulation report generated. Analogous to GitHub Actions pipelines, DevSpaceOps uses an "aerospace-grade test gate" to back open collaboration: it guarantees a quality floor for community contributions while greatly reducing manual testing cost, reconciling "openness" with "high reliability".

5.3 Shared Computing and Orchestration of the Simulation Platform

High-fidelity deep-space simulation is computationally demanding, and a single institution can hardly sustain it. This paper suggests organizing shared simulation capability as follows: (1) *distributed computing*: using cloud resource pools and donated public computing power to elastically scale large-scale simulation tasks on demand; (2) *federated simulation nodes*: different institutions maintain their own high-precision models and interconnect through standard interfaces, realizing "model federation, joint simulation" that protects each party's intellectual property while forming an overall verification capability; and (3) *result repository and benchmark suite*: publishing simulation results, benchmark scenarios and reference solutions to facilitate algorithm comparison, third-party review and community co-construction. The goal of shared computing organization is to enable teams of any size to obtain, at an affordable cost, verification capability approaching that of a national laboratory—an implementation of the "lowering the barrier" goal at the infrastructure level.

6 Security, Trust and Governance

6.1 Zero-Trust Architecture and Code Security

The code of the open-source aerospace ecosystem is contributed by people worldwide, and the risk is that unknown bugs, backdoors or malicious code may enter software that directly affects flight safety. Drawing on zero-trust architecture in cybersecurity, this paper advocates "never trust, always verify": any code, regardless of origin, must pass signature authentication, origin audit, static analysis, dependency scanning and sandboxed execution before entering the critical path; key software running on board should have hierarchical signing and integrity measurement (analogous to secure boot chains), and at runtime services should be isolated under the least-privilege principle, with each module accessing only the data and interfaces required by its duties. Zero trust decouples "contributor identity" from "code trustworthiness", so that the openness of the ecosystem need not come at the cost of security.

6.2 Byzantine Fault Tolerance and Distributed Consensus

On-orbit safety requires that the system still reach consensus and maintain operation even in the presence of partially failed, anomalous or malicious nodes. The distributed-systems field addresses this through the Byzantine generals problem and Byzantine fault tolerance (BFT) algorithms: under the assumption that "any node may be unreliable or malicious", redundancy and consensus algorithms guarantee consistency and correctness of system behavior. Transferring this to aerospace yields a "distributed deep-space computing network": redundant on-board computers cross-validate key commands, attitude decisions and state estimates through BFT consensus, so that even if individual computing units are affected

by software defects or attacks, the spacecraft still makes correct decisions based on majority agreement. Complementing this, the Safe Mode built into Space OS acts as a non-bypassable safety baseline that takes over automatically in the event of abnormal upper-layer code, guaranteeing minimal safeguards such as pointing toward the Sun, attitude stabilization and maintaining communication, thereby confining the risk introduced by open-source contributions to a controllable range.

6.3 Community Governance and Accountability

Security depends not only on technology but also on governance. The open-source aerospace ecosystem should clarify contribution agreements, license selection, code ownership and liability boundaries: first, uniformly adopt mature open-source licenses (e.g., Apache-2.0) with clear provisions on patent grants, commercial authorization and liability disclaimers; second, establish two-reviewer code review and dedicated review of high-sensitivity modules; third, clarify the division of responsibility that "the community is responsible for code quality and the deploying party is responsible for mission responsibility", and any institution that launches with open-source components must conduct its own mission-level safety assessment; and fourth, establish a security vulnerability disclosure and incident response mechanism (Security Advisory) with time-boxed fixes and full-network notifications for high-severity vulnerabilities.

6.4 The Tension Between Openness and Security

There is an objective tension between openness and security: fully opening the core safety logic may enable malicious parties to design attacks against it. This paper advocates determining the degree of openness by risk level: fully open general algorithms, simulation models, communication protocols and peripheral applications; for the flight-safety-critical trusted computing base, secure-boot keys and the Safe Mode implementation, adopt an "open design, controlled delivery" strategy—the design is public while build and deployment are controlled—analogous to the Kerckhoffs principle in cryptography that "the algorithm is public and the key is secret". Through graded openness, a balance is struck between maximizing ecosystem sharing and guarding the aerospace safety bottom line.

7 Business Models and Sustainability

7.1 Dual Licensing and Commercial Open Source

"Where does the money for open-source aerospace come from?" is the fundamental question of ecosystem viability. The dual-licensing and commercial-open-source practices of the computer industry (Red Hat, MySQL, Android) offer a mature answer: open source provides widely trusted infrastructure to attract participation, while commerce provides certainty, services and premium capabilities on top of it to generate profit. Mapped to aerospace, this

paper recommends: (1) *open-source base*: the Space OS core kernel, common simulation frameworks, basic algorithm libraries and open protocols are fully open for free use and contribution by global projects; (2) *commercial value-added layer*: commercial space companies (satellite operators, launch providers, startups) can, while using the open-source base, purchase high-precision physics simulation nodes, commercial-grade tracking, telemetry and control services, safety assessment and certification, mission customization and technical support; and (3) *service as revenue*: using the open-source base as an entry point and long-term service and professional support as the revenue source, forming the virtuous loop of "open source attracts, commerce feeds".

7.2 Diversified Funding Sources

The sustainability of the open-source aerospace ecosystem cannot rely on a single funding channel and should establish a diversified funding structure: (1) *foundations and donations*: following the Linux Foundation and Apache Foundation models, establish an aerospace open-source foundation that accepts corporate membership fees and individual donations; (2) *government funding*: include the open-source ecosystem in aerospace science and technology programs and pre-research projects, exchanging public input for ecosystem spillover benefits; (3) *corporate membership*: large aerospace groups, commercial space companies and supporting enterprises pay annual membership fees in exchange for governance seats and priority response to needs; and (4) *research-to-market conversion*: algorithms, models and standards produced in the ecosystem by universities and research institutes generate self-funding through technology-transfer mechanisms.

7.3 Industry–Academia–Research Collaboration and International Linkage

Open-source ecosystems are inherently cross-organizational and cross-border, and this should be fully exploited: universities are responsible for basic research, algorithm innovation and talent cultivation, embedding courses, theses and research projects into ecosystem development; research institutes and prime contractors provide mission traction, standard setting and safety oversight; and commercial space companies handle engineering, marketization and commercial feedback. Internationally, the ecosystem should actively engage with existing communities such as the NASA open-source program, ESA open toolchains and TinyGS, participate in international standard setting, and achieve complementary strengths while respecting each other's intellectual property and safety red lines, expanding the ecosystem's influence radius and intellectual supply.

8 Benchmark Cases and Feasibility Analysis

8.1 Benchmark Cases

The envisioned ecosystem is not fantasy; numerous comparable and verifiable precedents already exist in computing and aerospace. (1) *The NASA open-source software program.* NASA has long published open-source software and open data; its catalog covers orbital mechanics, mission planning and image processing, hosted on GitHub and open to the global community [15]. According to NASA's official statements, its Software Release Authority (SRA) has publicly released hundreds of software programs and code repositories. This proves that aerospace research institutions are willing and able to open core technological assets, and is the most direct real prototype of the "OpenSpace R&D ecosystem". (2) *The OpenSpace open-source visualization project.* Led by the American Museum of Natural History in New York, with participation from NASA, ESA and others, OpenSpace openly shares planetary science data as real-time 3D visualization [17], widely used for outreach and research. It shows that combining aerospace scientific data with rendering engines can form a sustainable open community. (3) *The open CubeSat ecosystem (TinyGS, ArduPilot, MAVLink).* TinyGS has built a globally distributed open network of LoRa satellite ground stations [18], where volunteers observe LEO satellites at minimal cost; ArduPilot and MAVLink provide widely used open-source flight-control firmware and software-in-the-loop (SITL) simulation capability [19, 20]. In terms of scale, by September 2024, 2,505 CubeSats had been launched worldwide, involving 88 countries and more than 600 organizations, with a record 390 launches in 2023 [27]; the commercial satellite industry generated about USD 285 billion in 2023, 71% of the global space economy [28]. These practices show that open-sourcing algorithms, firmware, ground stations and simulation tools has already been validated in real orbital missions, and that the prototype of a Space Package Registry already exists. (4) *Software-defined satellite and digital-twin engineering practice.* Multiple institutions in China and abroad have conducted research on software-defined satellites, on-orbit reconfiguration and digital-twin development models [23], verifying the technical feasibility of "platform-based, software-defined, simulation-first" development. (5) *Linux Foundation and other open-source governance models.* The organizational model of using foundations to host communities, licenses to regulate rights and membership fees to supply funding has been successfully validated by Linux, Apache, CNCF and others [3], and can be directly transferred to aerospace foundation design.

8.2 Feasibility Analysis

(1) *Technical feasibility*: digital-twin simulation, microservice architecture, containerized deployment, BFT consensus and zero-trust security are all mature general technologies, and hardware platforms (on-board computing, CubeSat platforms) are standardized; the obsta-

cles to transfer lie mainly in engineering adaptation rather than principle breakthroughs. (2) *Institutional feasibility*: the open-source licensing system is mature, and there are precedents for international openness of aerospace data and software; organizing cross-organization collaboration through foundations has adoptable legal and governance frameworks. Domestically, the reform direction of aerospace science and technology innovation and the construction of a new type of nationwide system provide policy space for the open-source ecosystem [22]. (3) *Economic feasibility*: open source lowers entry barriers and duplicated construction costs, commercial value-added services and diversified funding provide a self-sustaining mechanism, and shared simulation computing greatly amortizes verification costs. Industry data show that the global satellite industry already generated about USD 285 billion in revenue in 2023 with 9,691 active satellites in orbit [28], providing a solid demand and market base for ecosystem-based sharing. In the long run, the technological innovation and talent-cultivation benefits of the ecosystem significantly exceed its investment. (4) *Risks and responses*: the main risks include security risks (addressed by zero trust and graded openness), contribution-quality risks (addressed by DevSpaceOps test gates and review mechanisms), cold-start risks (insufficient early participants, mitigated by government traction projects, benchmark demonstrations and embedding in university curricula) and open-source sustainability risks (addressed by foundations and diversified funding). In summary, the proposed scheme is feasible along the technical, institutional and economic dimensions; the main bottleneck is the organizational mobilization and trust accumulation required in the initial stage.

9 Conclusion and Outlook

9.1 Main Conclusions

Taking the history of computer development as a reference frame, this paper studies the core question of how to build an open-source aerospace technology sharing ecosystem and a deep-space simulation platform, and reaches the following conclusions. First, the decades-long evolution of the computer industry contains deep architectural ideas that can be systematically transferred, distilled into six common laws—openness and sharing, standards first, layered decoupling, simulation first, security as a safety net and commercial feedback—which jointly answer how complex systems can be made efficient, reliable and usable by everyone. Data such as GitHub's 100 million developers, the millions of packages on npm/PyPI and the thousands of contributors to each Linux kernel release show that these laws have produced order-of-magnitude scale effects in the computer industry. Second, mapping these laws to aerospace yields an open-source aerospace technology sharing ecosystem consisting of community, standards, platform and application layers. The microservice-based spacecraft architecture and the universal spacecraft operating system (Space OS) decouple the

integrated satellite and alleviate the "one satellite, one customized system" problem; the Space Package Registry promotes component reuse through package management, significantly reducing duplicated construction cost. Third, the deep-space simulation platform is the key infrastructure for ecosystem implementation. A simulation system centered on the digital-twin environment, DevSpaceOps pipelines and shared computing transforms high-reliability verification from "building more prototypes and running more tests" to "automated extreme-condition testing", unifying open collaboration with high reliability. Fourth, security and sustainability are the two thresholds for ecosystem survival. Zero-trust architecture, Byzantine fault tolerance and graded openness protect the core safety baseline under distributed contributions; dual licensing and diversified funding provide self-sustaining capability. Benchmarking against existing practices such as NASA, OpenSpace and TinyGS, and the industrial scale of CubeSat growth from one thousand to two thousand in five years and the USD 285 billion satellite industry [27, 28], the proposed scheme is feasible along the technical, institutional and economic dimensions.

9.2 Limitations and Future Work

This paper is a conceptual and framework study whose argument rests mainly on historical comparison, case analysis and public statistics and has not yet been validated by engineering practice: the metric settings of the proposed concepts (e.g., the scale of "thousands of extreme-state tests"), the details of technical implementation and performance boundaries all await verification in prototype systems. Future work can deepen the research in the following directions: (1) build prototype systems of the digital twin and DevSpaceOps, validating technical feasibility with public datasets; (2) conduct a community pilot in a specific task scenario (e.g., a community for LEO satellite attitude-control algorithms) to test governance mechanisms and business models; (3) promote the drafting of open interface standards for aerospace, laying an institutional foundation for the ecosystem; and (4) complete the English version of this paper and its arXiv submission, bringing the research to a wider international academic and open-source community.

References

[1] Raymond E S. The Cathedral and the Bazaar: Musings on Linux and Open Source by an Accidental Revolutionary[M]. Sebastopol: O'Reilly Media, 1999.

[2] Stallman R. The GNU Manifesto[EB/OL]. (1985). https://www.gnu.org/gnu/manifesto.html.

[3] Fogel K. Producing Open Source Software: How to Run a Successful Free Software Project[M]. Sebastopol: O'Reilly Media, 2005.

[4] von Hippel E. Democratizing Innovation[M]. Cambridge: MIT Press, 2005.

[5] Chesbrough H W. Open Innovation: The New Imperative for Creating and Profiting from Technology[M]. Boston: Harvard Business School Press, 2003.

[6] Levy S. Hackers: Heroes of the Computer Revolution[M]. Sebastopol: O'Reilly Media, 1984.

[7] Bradner S. The Internet Standards Process[EB/OL]. RFC 2026, IETF, 1996.

[8] IEEE. Portable Operating System Interface (POSIX), IEEE Std 1003.1-2017[S]. New York: IEEE, 2017.

[9] Newman S. Building Microservices: Designing Fine-Grained Systems[M]. Sebastopol: O'Reilly Media, 2015.

[10] Kim G, Humble J, Debois P, et al. The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations[M]. Portland: IT Revolution Press, 2016.

[11] Merkel D. Docker: Lightweight Linux Containers for Consistent Development and Deployment[J]. Linux Journal, 2014(239): 2.

[12] Glaessgen E, Stargel D. The Digital Twin Paradigm for Future NASA and U.S. Air Force Vehicles[C]//53rd AIAA/ASME/ASCE/AHS/ASC Structures, Structural Dynamics and Materials Conference. Honolulu: AIAA, 2012: 1818.

[13] Grieves M, Vickers J. Digital Twin: Mitigating Unpredictable, Undesirable Emergent Behavior in Complex Systems[M]//Kahlen F J, Flumerfelt S, Alves A. Transdisciplinary Perspectives on Complex Systems. Cham: Springer, 2017: 85-113.

[14] Puig-Suari J, Turner C, Ahlgren W. Development of the Standard CubeSat Deployer and a CubeSat Class PicoSatellite[C]//2001 IEEE Aerospace Conference. Big Sky: IEEE, 2001: 1-347.

[15] NASA. NASA Open Source Software[EB/OL]. https://code.nasa.gov.

[16] European Space Agency. ESA Open Source[EB/OL]. https://github.com/esa.

[17] American Museum of Natural History, et al. OpenSpace[EB/OL]. https://www.openspaceproject.com.

[18] TinyGS. TinyGS: Open Ground Station Network[EB/OL]. https://tinygs.com.

[19] ArduPilot Project. ArduPilot: Open Source Autopilot[EB/OL]. https://ardupilot.org.

[20] MAVLink. MAVLink: Micro Air Vehicle Communication Protocol[EB/OL]. https://mavlink.io.

[21] Torvalds L, Hamano J. Git: The Distributed Version Control System[EB/OL]. https://git-scm.com.

[22] Information Office of the State Council of the People's Republic of China. China's Space Program: A 2021 Perspective[M]. Beijing: People's Publishing House, 2022.

[23] Xu F, Zhou X, Zhao J, et al. Conception and development of software-defined satellite technology[J]. Journal of Beijing University of Aeronautics and Astronautics, 2023, 49(7): 1543-1552 (in Chinese).

[24] GitHub. Octoverse 2024: The State of Open Source and Rise of AI in Software Development[EB/OL]. 2024. https://github.blog/news-insights/octoverse/octoverse-2024/.

[25] Sonatype. 2024 State of the Software Supply Chain[R]. 2024.

[26] Corbet J. Development Statistics for 6.3[EB/OL]. LWN.net, 2023. https://lwn.net/Articles/929582/.

[27] Kulu E. CubeSats & Nanosatellites — 2024 Statistics, Forecast and Reliability[C]//75th International Astronautical Congress (IAC 2024). 2024.

[28] Satellite Industry Association. State of the Satellite Industry Report 2024[R]. Washington, D.C.: SIA, 2024.